



**UNIVERSIDAD
NACIONAL DE
INGENIERÍA**



TRANSFORMACIÓN
digital

CURSOS GRATUITOS

OTI UNI

Fundamentos de Linux

Programa completo – 9na Edición

Instructor:  [Anthony Gonzalo Quispe Cordova](#)

Oficina de Tecnologías de la Información (OTI)

30 de mayo de 2026



Linux + Shell

Base para DevOps y automatización

Flujo de Cursos Orientado a DevOps



Horarios de los Cursos del Flujo DevOps



Fundamentos de Linux



Sábados



9:00 a.m. – 12:00 p.m.



23/05 – 27/06 | 18 h



Git y GitHub



Lun, Mar, Jue, Vie



4:00 – 7:00 p.m.



19/05 – 28/05 | 18 h



Fundamentos de Docker



Lun, Mar, Vie



4:00 – 7:00 p.m.



29/05 – 09/06 | 18 h



Ansible: Automatización



Sábados



2:00 – 5:00 p.m.



23/05 – 27/06 | 18 h

Índice General

Linux y Terminal desde Cero

- 1 Linux y su importancia actual
- 2 Ubuntu Server en VirtualBox
- 3 Terminal y comandos esenciales
- 4 Laboratorio practico

Usuarios, Permisos y Sistema de Archivos

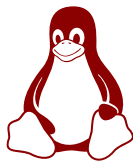
- 1 Identidad: usuarios, grupos y sudo
- 2 Permisos, propietarios y modos
- 3 Edicion de configuracion
- 4 Laboratorio multiusuario

Linux y Terminal desde Cero

Linux y su importancia actual

Que es Linux

- **Linux** es un sistema operativo libre, de codigo abierto y tipo Unix.
- Su componente central es el **kernel**: administra CPU, memoria, disco, red y dispositivos.
- Una **distribucion** agrega herramientas, instalador, repositorios y configuracion lista para usar.
- Ejemplos comunes: Ubuntu, Debian, Fedora, Rocky Linux, Kali Linux y Arch Linux.



Base de servidores modernos

Por que Linux domina servidores, nube y ciberseguridad

Servidores

Estabilidad, bajo consumo, automatización y administración remota por terminal.

Nube

La mayoría de servicios cloud, contenedores y pipelines DevOps trabajan sobre Linux.

Seguridad

Herramientas de análisis, hardening, monitoreo y respuesta suelen ejecutarse en Linux.

Aprender Linux es aprender la base operativa de infraestructura, DevOps, redes y seguridad.

Distribuciones: el mismo nucleo, distintos objetivos



Ubuntu Server

Servidores y aprendizaje.



Debian

Estabilidad y base comunitaria.



Rocky / RHEL

Empresa y producción.



Kali Linux

Seguridad ofensiva y laboratorios.

Familias de distribuciones Linux

Debian

- Debian
- Ubuntu
- Linux Mint
- Kali Linux

apt, dpkg

Red Hat

- RHEL
- Fedora
- Rocky Linux
- AlmaLinux

dnf, rpm

Arch / SUSE

- Arch Linux
- Manjaro
- openSUSE
- SUSE Linux

pacman, zypper

La familia define repositorios, formato de paquetes, comandos de instalacion y ciclo de actualizaciones.

Entornos de escritorio vs gestores de ventanas

Entorno de escritorio

- Experiencia grafica completa.
- Paneles, menus, configuracion y apps base.
- Ejemplos: GNOME, KDE Plasma, XFCE, Cinnamon.

Gestor de ventanas

- Controla como se abren y acomodan ventanas.
- Puede ser flotante o tipo mosaico.
- Ejemplos: i3, Openbox, AwesomeWM, Sway.

Importante para servidores

Ubuntu Server normalmente se administra **sin entorno grafico**; por eso la terminal es la herramienta principal.

Paqueterías y gestores de paquetes

Familia	Formato	Gestor	Ejemplo
Debian / Ubuntu	.deb	apt	<code>sudo apt install vim</code>
Red Hat / Fedora	.rpm	dnf	<code>sudo dnf install vim</code>
Arch / Manjaro	pkg.tar.zst	pacman	<code>sudo pacman -S vim</code>
openSUSE / SUSE	.rpm	zypper	<code>sudo zypper install vim</code>

Repositorio

Servidor confiable desde donde la distro descarga software y actualizaciones.

Paquete

Archivo instalable que incluye programa, metadatos, version y dependencias.

Formatos universales de instalacion

Snap

- Usado por Ubuntu.
- Apps autocontenidas.
- Comando: `snap`.

Flatpak

- Popular en escritorio.
- Repos como Flathub.
- Comando: `flatpak`.

AppImage

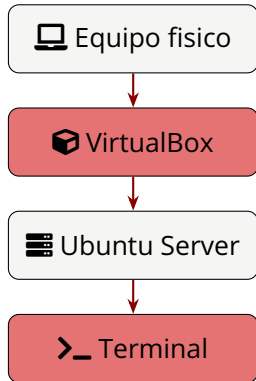
- Archivo ejecutable.
- No requiere instalacion tradicional.
- Util para apps portables.

En servidores se priorizan repositorios oficiales; en escritorio aparecen mas formatos universales.

Ubuntu Server en VirtualBox

Objetivo del entorno virtual

- Practicar Linux sin modificar el sistema principal.
- Crear un servidor real en una maquina virtual.
- Aprender a iniciar sesion y trabajar desde la terminal.
- Repetir, romper y reconstruir el laboratorio con seguridad.



Requisitos para la instalacion

Software

- Oracle VirtualBox instalado.
- ISO de Ubuntu Server.
- Conexion a internet para descargas.

Recursos sugeridos

- RAM: 2 GB como minimo.
- CPU: 2 nucleos si estan disponibles.
- Disco: 20 GB dinamico.
- Red: NAT para empezar.

Antes de iniciar

Verifica que la virtualizacion este habilitada en BIOS/UEFI si VirtualBox no permite crear maquinas de 64 bits.

Crear la maquina virtual

- 1 Crear una nueva VM: **Linux / Ubuntu (64-bit)**.
- 2 Asignar memoria y procesadores.
- 3 Crear disco duro virtual VDI, reservado dinamicamente.
- 4 Montar la ISO de Ubuntu Server en la unidad optica.
- 5 Iniciar la VM y seguir el instalador.

Meta: terminar con un servidor Ubuntu arrancando y pidiendo usuario/contrasena en consola.

Decisiones dentro del instalador

Configuracion base

- Idioma y teclado.
- Red por DHCP.
- Disco completo para la VM.
- Usuario administrador del laboratorio.

Perfil recomendado

- Nombre servidor: `linux-lab`.
- Usuario: `pit`.
- Instalar OpenSSH si se usara acceso remoto.
- No instalar extras innecesarios.

Primer ingreso al servidor

- Inicia sesion con el usuario creado durante la instalacion.
- Confirma que estas en Linux y que tienes conexion.
- Ubica tu carpeta personal y reconoce el prompt.

Comandos iniciales

```
whoami  
hostname  
pwd  
ip addr
```

Terminal y comandos esenciales

Terminal, shell y sistema de archivos

- **Terminal:** interfaz donde escribimos comandos.
- **Shell:** programa que interpreta esos comandos; en Ubuntu suele ser Bash.
- **Ruta absoluta:** empieza desde la raiz, por ejemplo `/home/pit`.
- **Ruta relativa:** parte desde la ubicacion actual, por ejemplo `Documentos/notas.txt`.

En Linux todo vive dentro de un arbol que empieza en `/`; no existen unidades `C:` o `D:`.

Estructura basica del sistema

Directorio	Uso principal
/	Raiz del sistema
/home	Carpetas de usuarios
/etc	Configuracion del sistema
/var	Datos variables como logs
/tmp	Archivos temporales
/bin	Comandos esenciales

La primera practica se concentrara en moverse por este arbol y crear estructura propia.

Comandos para navegar

Ubicacion

```
pwd  
ls  
ls -la
```

Movimiento

```
cd carpeta  
cd ..  
cd ~  
clear
```

Comandos para crear y gestionar

Crear

```
mkdir proyecto  
mkdir -p app/src  
touch notas.txt  
nano notas.txt
```

Gestionar

```
cp origen destino  
mv origen destino  
rm archivo  
rm -r carpeta
```

Precaucion

Antes de eliminar con `rm`, confirma tu ruta con `pwd` y revisa con `ls`.

Ver contenido de archivos

Lectura rapida

```
cat archivo  
head archivo  
tail archivo  
less archivo
```

Buenas practicas

- Usar nombres claros.
- Evitar espacios en rutas al empezar.
- Organizar por carpetas.
- Leer mensajes de error con calma.

Laboratorio pratico

Laboratorio: estructura de proyecto en Linux

- Crear una carpeta principal del curso.
- Separar documentacion, scripts, datos y respaldos.
- Crear archivos iniciales.
- Copiar, mover y renombrar elementos.
- Validar la estructura final desde terminal.

Estructura esperada

```
proyecto-linux/  
  docs/  
  scripts/  
  data/  
  backups/  
  README.txt
```

Paso 1: crear carpetas y archivos

Comandos

```
cd ~  
mkdir -p proyecto-linux/{docs,scripts,data,backups}  
cd proyecto-linux  
touch README.txt docs/notas.txt scripts/inicio.sh  
pwd  
ls -la
```

La opcion -p permite crear carpetas intermedias sin error si ya existen.

Paso 2: escribir y revisar contenido

Comandos

```
echo "Proyecto Linux - Bloque inicial" > README.txt
echo "Comandos practicados:" > docs/notas.txt
echo "pwd, ls, cd, mkdir, touch" >> docs/notas.txt
cat README.txt
cat docs/notas.txt
```

En esta semana solo usaremos escritura basica para documentar el laboratorio.

Paso 3: copiar, mover y validar

Comandos

```
cp README.txt backups/README.bak  
mv docs/notas.txt docs/comandos-basicos.txt  
ls -R
```

Resultado: un proyecto ordenado, navegable y documentado desde la terminal.

Checklist de aprendizaje

- Puedo explicar que es Linux y por que se usa en servidores.
- Puedo diferenciar distribuciones, escritorios y gestores de paquetes.
- Puedo describir como se instala Ubuntu Server en VirtualBox.
- Puedo moverme por el sistema de archivos desde la terminal.
- Puedo crear, copiar, mover y revisar archivos con comandos basicos.

Cierre: Linux y Terminal

- 1 Linux es la base de servidores, nube, DevOps y ciberseguridad.
- 2 Ubuntu Server en VirtualBox permite practicar en un entorno real y controlado.
- 3 La terminal y Bash permiten controlar el sistema mediante comandos.
- 4 El sistema de archivos parte desde / y organiza todo en carpetas.
- 5 El laboratorio construyo una estructura de proyecto real en Linux.

Siguiente bloque

Tema siguiente: usuarios, permisos y sistema de archivos. Veremos como Linux organiza el acceso y protege recursos del sistema.

Identidad: usuarios, grupos y sudo

Objetivos: usuarios, permisos y archivos

- Entender como Linux controla el acceso mediante usuarios, grupos y permisos.
- Leer permisos y propietarios sin depender de memoria mecanica.
- Usar sudo, chmod, chown y chgrp con criterio.
- Editar archivos de configuracion con Nano y una base practica de Vim.
- Construir un entorno multiusuario con accesos protegidos por rol.



**Seguridad por
identidad, propiedad y
permisos**

Como Linux decide quien puede hacer que

Usuario

Identidad que ejecuta procesos, crea archivos y tiene una carpeta de trabajo.

Grupo

Conjunto de usuarios usado para compartir permisos sobre recursos.

Otros

Cualquier usuario que no sea propietario ni pertenezca al grupo del recurso.

Cada archivo tiene propietario, grupo propietario y permisos separados para usuario, grupo y otros.

Usuarios, root y sudo

Identidades clave

- **root:** administrador total del sistema.
- **Usuario regular:** trabaja con permisos limitados.
- **sudo:** eleva privilegios solo para una accion concreta.

Comandos de reconocimiento

```
whoami  
id  
groups  
sudo -l
```

Regla operativa

Operar como usuario normal y elevar privilegios solo cuando sea necesario.

Gestion basica de cuentas y grupos

Usuarios

```
sudo adduser ana
sudo passwd ana
id ana
sudo deluser ana
```

Grupos

```
sudo groupadd devops
sudo usermod -aG devops ana
groups ana
sudo groupdel devops
```

Usar `usermod -aG`: `-a` agrega sin borrar otros grupos; `-G` define grupos suplementarios.

Archivos del sistema relacionados con identidad

Archivo	Funcion principal
<code>/etc/passwd</code>	Lista usuarios, UID, GID principal, carpeta personal y shell.
<code>/etc/shadow</code>	Guarda hashes de contraseñas y políticas de expiración.
<code>/etc/group</code>	Define grupos y miembros suplementarios.
<code>/etc/sudoers</code>	Controla quien puede ejecutar comandos con privilegios.

Cuidado

No editar `/etc/sudoers` con un editor comun. Usar `visudo` para validar sintaxis antes de guardar.

Permisos, propietarios y modos

Leer permisos con ls -l

Salida típica

```
-rw-r----- 1 ana devops 1240 may 30 14:10 reporte.txt
drwxr-x--- 2 root devops 4096 may 30 14:15 proyectos
```

- Primer caracter: tipo de archivo.
- Luego vienen permisos de usuario, grupo y otros.
- Después aparecen propietario y grupo.
- Fechas, tamaños y nombres ayudan a validar cambios.

Permisos rwx

r lectura

Permite leer un archivo o listar nombres dentro de un directorio.

w escritura

Permite modificar un archivo o crear, borrar y renombrar dentro de un directorio.

x ejecucion

Permite ejecutar un archivo o atravesar un directorio para acceder a su contenido.

En directorios, x significa acceso/travesia; sin x, listar no basta para entrar.

Modo simbolico y modo octal

Simbolico

- u: usuario propietario.
- g: grupo propietario.
- o: otros.
- a: todos.
- Operadores: +, -, =.

Octal

- r = 4
- w = 2
- x = 1
- 755 = `rwX r-X r-X`
- 640 = `rw- r- --`

chmod: cambiar permisos

Modo simbolico

```
chmod u+x script.sh
chmod g+w reporte.txt
chmod o-r secreto.txt
chmod a=r archivo.txt
```

Modo octal

```
chmod 755 script.sh
chmod 644 index.html
chmod 640 reporte.txt
chmod 700 carpeta_privada
```

Evitar

`chmod 777` no debe ser la solución por defecto: abre lectura, escritura y ejecución a todos.

chown y chgrp: cambiar propiedad

Propietario y grupo

```
sudo chown ana reporte.txt  
sudo chown ana:devops reporte.txt  
sudo chown -R ana:devops proyecto/
```

Solo grupo

```
sudo chgrp devops reporte.txt  
sudo chgrp -R devops proyecto/  
ls -l
```

Cambiar propiedad requiere privilegios porque modifica quien controla el recurso.

Permisos recomendados en situaciones comunes

Caso	Permiso	Criterio
Archivo publico de lectura	644	Propietario escribe; todos leen.
Script ejecutable	755	Propietario modifica; otros ejecutan.
Archivo sensible	600	Solo propietario lee y escribe.
Carpeta privada	700	Solo propietario entra y modifica.
Carpeta compartida por grupo	770	Usuario y grupo trabajan; otros no entran.

El permiso correcto depende del rol del archivo, no de una receta unica.

Enlaces simbolicos y enlaces duros

Enlace simbolico

Apunta a una ruta. Si el destino cambia o se borra, el enlace puede quedar roto.

```
ln -s destino enlace
```

Enlace duro

Comparte el mismo inodo del archivo. No es una copia separada.

```
ln destino enlace
```

Para administracion diaria y configuraciones, el enlace simbolico suele ser el mas visible y facil de interpretar.

Edicion de configuracion

Nano: edicion directa para empezar

- Simple para editar archivos pequenos.
- Muestra atajos en la parte inferior.
- Es util cuando el objetivo es corregir rapido y guardar.

Flujo basico

```
nano archivo.conf
Ctrl + O    guardar
Enter      confirmar nombre
Ctrl + X    salir
```


Vim basico: sobrevivir en servidores

Modos esenciales

```
vim archivo.conf
i          insertar
Esc        modo normal
:w         guardar
:q         salir
:wq        guardar y salir
:q!       salir sin guardar
```

Acciones utiles

```
/texto    buscar
dd         borrar linea
u          deshacer
yy         copiar linea
p          pegar
```

Buenas practicas al editar configuracion

- 1 Identificar el archivo exacto y confirmar ruta con `pwd` o ruta absoluta.
- 2 Hacer copia antes de cambiar: `sudo cp archivo archivo.bak`.
- 3 Editar el minimo necesario y guardar con comentarios claros si aplica.
- 4 Validar sintaxis cuando exista comando de prueba.
- 5 Reiniciar o recargar el servicio solo despues de validar.

Criterio

En servidores, editar sin copia y sin validacion convierte un cambio pequeno en una caida innecesaria.

Laboratorio multiusuario

Laboratorio: entorno multiusuario por rol

- Crear usuarios para simular un equipo de trabajo.
- Crear grupos por rol.
- Preparar una carpeta compartida.
- Asignar propietario, grupo y permisos.
- Validar que cada usuario solo acceda a lo que corresponde.

Escenario

Equipo: devops
Usuarios: ana, luis
Carpeta: /srv/proyecto
Acceso: grupo puede trabajar; otros no.

Paso 1: crear usuarios, grupo y carpeta

Preparar

```
sudo adduser ana
sudo adduser luis
sudo groupadd devops
sudo usermod -aG devops ana
sudo usermod -aG devops luis
sudo mkdir -p /srv/proyecto
```

En un laboratorio real se pueden usar contraseñas temporales y luego eliminarlas al finalizar.

Paso 2: asignar propiedad y permisos

Configurar

```
sudo chown root:devops /srv/proyecto  
sudo chmod 770 /srv/proyecto  
ls -ld /srv/proyecto
```

Interpretacion

770 permite acceso total al propietario y al grupo; bloquea completamente a otros usuarios.

Paso 3: validar accesos

Usuario autorizado

```
su - ana  
cd /srv/proyecto  
touch ana.txt  
ls -l  
exit
```

Revision como administrador

```
sudo ls -la /srv/proyecto  
sudo -u luis bash  
touch /srv/proyecto/luis.txt  
exit  
sudo ls -la /srv/proyecto
```

Paso 4: limpiar el laboratorio

Limpieza

```
sudo rm -rf /srv/proyecto  
sudo deluser ana  
sudo deluser luis  
sudo groupdel devops
```

Antes de borrar

Confirmar la ruta exacta con `ls -ld /srv/proyecto`. Nunca ejecutar `rm -rf` sobre una ruta dudosa.

Checklist de aprendizaje

- Puedo explicar la diferencia entre usuario, grupo y otros.
- Puedo leer permisos desde una salida `ls -l`.
- Puedo aplicar `chmod` en modo simbolico y octal.
- Puedo cambiar propietario y grupo con `chown` y `chgrp`.
- Puedo editar configuraciones basicas con Nano y salir correctamente de Vim.
- Puedo construir una carpeta compartida protegida por rol.

Resumen: usuarios, permisos y archivos

- 1 Linux separa identidad, propiedad y permisos para proteger recursos.
- 2 `sudo` permite privilegios temporales sin trabajar siempre como `root`.
- 3 `chmod` define accesos; `chown` y `chgrp` definen control.
- 4 Nano y Vim permiten modificar configuraciones desde servidores sin entorno grafico.
- 5 El laboratorio consolida un entorno multiusuario con acceso por rol.

Siguiente bloque

Tema siguiente: procesos, servicios y paquetes. Veremos como monitorear el sistema, controlar procesos y administrar software.